🎙️ Transcribing Videos with yt-dlp and faster-whisper

Fulllion Creative Works

A step-by-step guide to turning videos into timestamped text on your own PC, with yt-dlp to fetch the audio and faster-whisper to transcribe it.

Throughout, replace:

Placeholder	With
`<video-url>`	the address of a video you're allowed to download
`<your-folder>`	the folder that holds your audio or video files
`C:\Tools\yt-dlp` and `C:\Tools\faster-whisper`	wherever you keep your tools, if you prefer somewhere else

Contents

1. How It Fits Together
2. Before You Start
3. Install yt-dlp
4. Download Audio Only
5. Install faster-whisper
6. Transcribe One File
7. Choose a Model
8. CPU or GPU, and How Fast
9. Transcribe a Whole Folder
10. Run at Low Priority
11. Privacy and Copyright
12. Troubleshooting
13. Sources

---

1. How It Fits Together

Tool	What it does
yt-dlp	A command-line downloader for YouTube and thousands of other sites. Here it fetches only the audio track, which is much smaller than the video.
ffmpeg	A media converter. yt-dlp uses it to merge and convert files. You can skip it for this guide (Section 4.4).
faster-whisper	A fast version of OpenAI's Whisper speech recognition model. It runs on your own CPU or NVIDIA GPU.

Tool	What it does
PyAV	The library faster-whisper uses to read audio. It includes its own copy of FFmpeg's libraries.

The flow is:

```
Video page ──yt-dlp, audio only──▶ talk.m4a ──faster-whisper──▶ talk.txt
```

Everything after the download happens on your PC. faster-whisper downloads its model from Hugging Face once, the first time you use it, and then works offline. Your audio never leaves your machine.

Already have the video files, such as your own recordings? Skip Sections 3 and 4.

---

2. Before You Start

You'll need:

- **Windows 10 or 11.** macOS and Linux work too. Where the commands differ, it's noted.
- **Python 3.11 or newer.** yt-dlp needs 3.11 or newer, and faster-whisper needs 3.9 or newer. To install Python, see Python and Node Tool Environments, or the winget section of the CLI Cheat Sheet.
- **A few GB of free disk space** for the tools and models, more if you use the large models.
- **Videos you're allowed to use** (Section 11).

Each tool gets its own folder with its own **virtual environment** (venv): a private copy of Python and its packages. This keeps the tools from clashing with each other or with other Python projects, and you can delete a folder to remove a tool completely. Python and Node Tool Environments explains venvs in more detail, and its Section 10 shows this one-folder-per-tool layout.

---

3. Install yt-dlp

1. Open **PowerShell** and make a folder with a venv inside it:

   ```
   mkdir C:\Tools\yt-dlp
   cd C:\Tools\yt-dlp
   py -m venv .venv
   ```

2. Turn the venv on for this window:

   ```
   .\.venv\Scripts\Activate.ps1
   ```

   Your prompt now starts with `(.venv)`. If PowerShell refuses to run the script, see Python and Node Tool Environments, or skip this step and type the full path instead (Section 6 of that guide explains how), such as `C:\Tools\yt-dlp\.venv\Scripts\yt-dlp.exe`.

3. Install yt-dlp with its recommended extras:

   ```
   python -m pip install -U "yt-dlp[default]"
   ```

4. Check it works:

```
yt-dlp --version
```

On macOS and Linux, use `python3 -m venv .venv`, then `source .venv/bin/activate`.

Before installing any package, it's worth a quick check that it's the real one. Safer Package Installs with npm and pip shows how. yt-dlp's PyPI name is exactly `yt-dlp`.

> **Other ways to install.** yt-dlp also comes as a standalone `yt-dlp.exe` on its GitHub releases page, and through winget (`winget install --source winget --id yt-dlp.yt-dlp`). This guide uses the venv, so updates and removal stay in one folder.

3.1 YouTube needs a JavaScript runtime

For full YouTube support, yt-dlp needs two more things:

- **yt-dlp-ejs**, a set of helper scripts. The `[default]` extra in step 3 already installed it.
- **A JavaScript runtime** to run them. **Deno** is recommended, and it's the only one yt-dlp uses without being told. Install it from Deno's installation page.

Already have **Node.js** 22 or newer? You can use it instead by adding `--js-runtimes node` to your yt-dlp commands. To install Node, see Section 1 of Building Firebase Apps in Google Antigravity IDE.

Other sites don't need this.

3.2 Keep it updated

Sites change often, and an old yt-dlp is the most common reason a download stops working. To update, run the install command from step 3 again, with the venv turned on:

```
python -m pip install -U "yt-dlp[default]"
```

The built-in `yt-dlp -U` updater is for the standalone `.exe`. For a pip install, yt-dlp's README says to re-run the command you installed it with.

---

4. Download Audio Only

Turn the venv on first (Section 3, step 2), then go to the folder where you want the audio saved:

```
cd "<your-folder>"
```

4.1 Check what you're getting

`--print` (short form `-O`) shows details about a video without downloading it:

```
yt-dlp --no-playlist -O title -O duration_string -O uploader "<video-url>"
```

Printing implies **simulate**, so nothing is saved. To see every format the site offers, with its file type and size:

```
yt-dlp --no-playlist -F "<video-url>"
```

4.2 Download the audio

```
yt-dlp -f "bestaudio[ext=m4a]" --no-playlist -o "talk.%(ext)s" "<video-url>"
```

Part	What it does
`-f "bestaudio[ext=m4a]"`	Picks the best **audio-only** format whose file type is `m4a`. No video is downloaded.
`--no-playlist`	Downloads only this video, even if the link is part of a playlist.
`-o "talk.%(ext)s"`	Names the file. `%(ext)s` is replaced with the real file type, so this saves `talk.m4a`.
`"<video-url>"`	Keep the quotes. Links often contain `&`, which the terminal would otherwise treat as a command.

If a video has no `m4a` audio, add a fallback after a `/`. yt-dlp tries each choice in order:

```
yt-dlp -f "bestaudio[ext=m4a]/bestaudio" --no-playlist "<video-url>"
```

The fallback is often a `.webm` file, which faster-whisper also reads.

4.3 Name files with templates

`-o` takes a **template**: text with fields in `%(...)s` that yt-dlp fills in. Without `-o`, yt-dlp uses `%(title)s [%(id)s].%(ext)s`, such as `My Talk [dQw4w9WgXcQ].m4a`.

Template	Gives
`"%(title)s.%(ext)s"`	`My Talk.m4a`
`"%(upload_date)s %(title)s.%(ext)s"`	`20261104 My Talk.m4a` (upload date as YYYYMMDD)
`"%(uploader)s/%(title)s.%(ext)s"`	A folder per channel

Two related options:

- **`-P "<your-folder>\Audio"`** sets the folder to save into, so you don't have to `cd` there first.
- **`--restrict-filenames`** keeps names to plain ASCII, with no spaces or `&`. It's handy if other tools choke on unusual characters.

> **In a `.bat` file, write `%%` for every `%`.** Command Prompt treats `%` specially in batch files, so `%(title)s` becomes `%%(title)s`. Typed directly in PowerShell or Command Prompt, a single `%` is fine.

4.4 When you need ffmpeg

yt-dlp's README calls ffmpeg "strongly recommended", because it's needed for:

- **Merging** a separate video stream and audio stream into one file. Sites such as YouTube often serve the best video and audio as separate streams, so downloading a full video in good quality usually needs ffmpeg.
- **Converting** audio with `-x` (`--extract-audio`), `--audio-format mp3` or the `-t mp3` preset.
- **Fixing up** some files after download, such as YouTube's DASH `.m4a` files.

For transcription, you need none of these. Picking an audio-only format with `-f`, as in Section 4.2, downloads one ready-to-use file. Without ffmpeg, yt-dlp prints a warning and gives you the format as it is, which faster-whisper reads fine.

If you do want ffmpeg, note that it's a separate program, **not** the Python package called `ffmpeg`. yt-dlp's README links to builds, and `--ffmpeg-location` tells yt-dlp where you put it.

4.5 Playlists and channels

A link with `&list=` in it belongs to a playlist. `--no-playlist` downloads just the one video. To download a whole playlist you're allowed to use, leave it off and add an archive file, so a re-run skips what's already done:

```
yt-dlp -f "bestaudio[ext=m4a]" --download-archive done.txt "<playlist-url>"
```

yt-dlp records each video's ID in `done.txt` and skips those IDs next time.

4.6 Your own YouTube videos

If you uploaded the video, you don't need yt-dlp. In **YouTube Studio**, go to **Content**, open the video's **Options** menu (three dots) and choose **Download**. It gives you an MP4 at 720p or 360p, depending on the video. **Google Takeout** can download all your uploads at once.

---

5. Install faster-whisper

1. Open a **new** PowerShell window, so the yt-dlp venv isn't on, and make a second folder:

   ```
   mkdir C:\Tools\faster-whisper
   cd C:\Tools\faster-whisper
   py -m venv .venv
   .\.venv\Scripts\Activate.ps1
   ```

2. Install faster-whisper, with PyAV held below version 19:

   ```
   python -m pip install faster-whisper "av<19"
   ```

3. Check it imports:

   ```
   python -c "import faster_whisper; print(faster_whisper.__version__)"
   ```

You don't need to install ffmpeg. PyAV includes FFmpeg's libraries, so faster-whisper can read `.m4a`, `.mp3`, `.mp4`, `.mkv`, `.webm` and other common files on its own.

> **Why `"av<19"`?** PyAV 19.0.0 removed an option (`metadata_errors`) that faster-whisper 1.2.1 passes every time it opens a file, so opening any file fails with a `TypeError`. faster-whisper's maintainers merged a fix in September 2026, but as of November 2026 no release includes it yet. Holding PyAV below 19 avoids the problem. Once a newer faster-whisper is out, you can drop the pin. The Section 9 script also sidesteps it by reading the audio itself.

---

6. Transcribe One File

6.1 The script

Save this as `C:\Tools\faster-whisper\transcribe.py`:

```
"""Transcribe one audio or video file to timestamped text."""
import sys
from pathlib import Path

from faster_whisper import WhisperModel


def stamp(seconds):
    m, s = divmod(int(seconds), 60)
    h, m = divmod(m, 60)
    return f"{h}:{m:02d}:{s:02d}"


source = Path(sys.argv[1])
target = source.with_suffix(".txt")

model = WhisperModel("distil-small.en", device="cpu", compute_type="int8")
segments, info = model.transcribe(str(source), language="en", vad_filter=True)

with target.open("w", encoding="utf-8") as out:
    for seg in segments:
        line = f"[{stamp(seg.start)} -> {stamp(seg.end)}] {seg.text.strip()}"
        print(line)
        out.write(line + "\n")

print(f"Saved {target} ({stamp(info.duration)} of audio)")
```

What it does:

- **`WhisperModel("distil-small.en", device="cpu", compute_type="int8")`** loads a small, fast English model on the CPU. `int8` stores the model's numbers in a compact form, which is faster on CPUs. Section 7 covers other models.

- `transcribe(...)` returns the `segments` (stretches of speech, each with a start and end time) and `info` (such as the audio's length).
- `language="en"` skips language detection. Remove it if you switch to a multilingual model (Section 7).
- `vad_filter=True` skips silences longer than about 2 seconds, which saves time and cuts down on made-up text in quiet parts.
- `segments` **is a generator.** The real work happens as the loop runs, which is why lines appear a few at a time.

6.2 Run it

With the faster-whisper venv on:

```
cd C:\Tools\faster-whisper
python transcribe.py "<your-folder>\talk.m4a"
```

It writes `talk.txt` next to the audio file, with lines like:

```
[0:00:04 -> 0:00:09] Thanks for coming, everyone. Today I want to talk about...
```

6.3 The first run downloads the model

The first time you use a model, faster-whisper downloads it from the **Hugging Face Hub**. That can take a few minutes, and nothing prints in the meantime. Later runs load it from your disk and don't need the internet.

Where it's stored	`%USERPROFILE%\.cache\huggingface\hub` on Windows, `~/.cache/huggingface/hub` on macOS and Linux, in folders like `models--Systran--faster-distil-whisper-small.en`
To store it elsewhere	Set the `HF_HOME` or `HF_HUB_CACHE` environment variable, or pass `download_root=r"D:\Models"` to `WhisperModel`
To stay offline	Pass `local_files_only=True` to `WhisperModel`. It then fails instead of trying to download
To free space	Delete the model's `models--...` folder. It downloads again if you use it later

On Windows, you may see a warning that the cache can't use **symlinks**. It's harmless: files are copied instead, which uses a little more space. Turning on Windows **Developer Mode** removes it, or you can hide it by setting `HF_HUB_DISABLE_SYMLINKS_WARNING` to `1`.

---

7. Choose a Model

Bigger models are more accurate and slower. Pass the name as the first argument to `WhisperModel`.

Model	Languages	Good for
`tiny`, `base`	Many	Quick drafts and slow PCs. Noticeably less accurate.
`tiny.en`, `base.en`	English only	The same, but more accurate for English.

Model	Languages	Good for
`small`, `small.en`	Many / English	A good balance on a CPU.
`medium`, `medium.en`	Many / English	Better accuracy. Slow on a CPU.
`large-v3` (or `large`)	Many	The most accurate. Best on a GPU.
`turbo` (`large-v3-turbo`)	Many	Close to large-v3, much faster. OpenAI notes it isn't trained for translation.
`distil-small.en`	English only	Fast on a CPU, with accuracy close to larger models. A good default for English.
`distil-medium.en`, `distil-large-v3`	English only	More accurate distil models, still faster than the originals.

- **The `.en` models only do English,** and OpenAI's notes say they tend to do better than the multilingual ones at English, especially `tiny.en` and `base.en`.
- **The distil models** are "distilled" versions of Whisper: smaller and several times faster, with accuracy within about 1% of the originals on the project's tests. Distil-Whisper is English only.
- **For other languages,** use a multilingual model and either set `language` (such as `language="fr"`) or leave it out to detect it automatically.

Where to start: `distil-small.en` for English on a CPU. If it gets too many words wrong, try `distil-large-v3` or `medium.en`. With an NVIDIA GPU, go straight to `large-v3` or `turbo`.

---

8. CPU or GPU, and How Fast

8.1 CPU

`device="cpu", compute_type="int8"` works on any PC and is what this guide uses. To choose how many CPU threads it uses, pass `cpu_threads`, such as `WhisperModel(..., cpu_threads=8)`. Fewer threads leave more of the PC free for other work.

8.2 NVIDIA GPU

On an NVIDIA graphics card, change one line:

```
model = WhisperModel("large-v3", device="cuda", compute_type="float16")
```

`compute_type="int8_float16"` uses less video memory. The GPU route needs NVIDIA's libraries as well as the graphics driver:

- **cuBLAS for CUDA 12** and **cuDNN 9 for CUDA 12.** The current CTranslate2 (the engine underneath faster-whisper) supports only these.
- **On older CUDA versions,** faster-whisper's README says to install an older CTranslate2: `3.24.0` for CUDA 11 with cuDNN 8, or `4.4.0` for CUDA 12 with cuDNN 8.

- **On Windows,** install them from NVIDIA's own CUDA and cuDNN downloads, and make sure their `bin` folders are on your `PATH`. faster-whisper's README also points to a third-party archive of the libraries (Purfview's whisper-standalone-win). Check where anything comes from before you run it.
- **On Linux,** the README shows installing them with pip (`nvidia-cublas-cu12` and `nvidia-cudnn-cu12==9.*`) and setting `LD_LIBRARY_PATH`.

faster-whisper's GPU support is for NVIDIA cards through CUDA. On AMD or Intel graphics, or a Mac, use the CPU settings.

8.3 How fast to expect

- **faster-whisper's own benchmark:** 13 minutes of audio, the `small` model with `int8` on 8 threads of an Intel Core i7-12700K, took 1 minute 42 seconds. That's about 7.5 times faster than real time.
- **A typical desktop CPU:** `distil-small.en` with `int8` runs at about **8 times real time**, so an hour of audio takes around 8 minutes.
- **On a GPU,** the large models run many times faster than on a CPU. The README's benchmark ran large-v2 on an RTX 3070 Ti in about a minute for the same 13 minutes.

Speed depends on your processor, the model, and how much talking there is. To go faster on a CPU, use a smaller or distil model, keep `int8`, or pass `beam_size=1` to `transcribe` (the default is 5), which trades a little accuracy for speed. Test on a short file before starting a long batch.

---

9. Transcribe a Whole Folder

This script transcribes every audio and video file in a folder into a `Transcripts` folder beside them. It **skips files that already have a transcript**, so you can stop it at any time and run it again to carry on. It reads the audio with PyAV itself, so it works with any PyAV version.

Save it as `C:\Tools\faster-whisper\transcribe_folder.py`:

```
"""Transcribe every audio or video file in a folder, skipping finished ones.

Transcripts are the speaker's own words: keep them private.
"""
import sys
from pathlib import Path

import av
import numpy as np
from faster_whisper import WhisperModel

MODEL = "distil-small.en"
MEDIA = {".m4a", ".mp3", ".wav", ".flac", ".ogg", ".opus", ".webm",
         ".mp4", ".mkv", ".mov", ".avi"}


def decode(path):
    """Read the first audio track as 16 kHz mono, as Whisper expects."""
    resampler = av.AudioResampler(format="s16", layout="mono", rate=16000)
    chunks = []
    with av.open(str(path)) as container:
        stream = container.streams.audio[0]
        for frame in container.decode(stream):
            for out in resampler.resample(frame):
                chunks.append(out.to_ndarray().reshape(-1))
        for out in resampler.resample(None):
            chunks.append(out.to_ndarray().reshape(-1))
    return np.concatenate(chunks).astype(np.float32) / 32768.0


def stamp(seconds):
    m, s = divmod(int(seconds), 60)
    h, m = divmod(m, 60)
    return f"{h}:{m:02d}:{s:02d}"


def main():
    folder = Path(sys.argv[1])
    out_dir = folder / "Transcripts"
    out_dir.mkdir(exist_ok=True)
    files = sorted(p for p in folder.iterdir() if p.suffix.lower() in MEDIA)
    print(f"{len(files)} file(s) found")

    model = WhisperModel(MODEL, device="cpu", compute_type="int8")
    for path in files:
        target = out_dir / (path.stem + ".txt")
        if target.exists():
            print(f"skip (already done): {path.name}")
            continue
        print(f"transcribing: {path.name}")
        try:
            segments, info = model.transcribe(
                decode(path), language="en", vad_filter=True,
                condition_on_previous_text=False)
```

```
            lines = [f"# {path.name} (private transcript, not for sharing)"]
            lines += [f"[{stamp(s.start)}] {s.text.strip()}" for s in segments]
        except Exception as error:
            print(f"FAILED: {path.name}: {error}")
            continue
        part = target.with_suffix(".part")
        part.write_text("\n".join(lines) + "\n", encoding="utf-8")
        part.replace(target)
        print(f"done: {path.name} ({stamp(info.duration)} of audio)")


if __name__ == "__main__":
    main()
```

Run it with the faster-whisper venv on:

```
cd C:\Tools\faster-whisper
python transcribe_folder.py "<your-folder>"
```

How it stays safe to stop and restart:

- **It writes to a `.part` file first,** then renames it to `.txt` only when the file is finished. A stopped run never leaves a half-written transcript that would be skipped next time.
- **A file that fails** is reported and skipped, and the rest carry on. Fix the problem and run it again, and only the missing ones are done.
- **To redo a file,** delete its `.txt` from `Transcripts`.
- **`condition_on_previous_text=False`** stops each segment from being guided by the last one. That makes the model less likely to get stuck repeating a phrase in a long recording.

> **Using a script that only looks for video extensions?** A `.m4a` file renamed to `.mp4` still works, because both are the same MP4 container. Adding `.m4a` to the script's list, as above, is the tidier fix.

---

10. Run at Low Priority

Transcription keeps your CPU busy. Run it at a lower priority and Windows gives the processor to everything else first, so your PC stays responsive and the job uses whatever is left over.

10.1 Command Prompt

`start /low` runs a program at **Low** priority (Windows calls it **Idle**):

```
cd /d C:\Tools\faster-whisper
start "" /low /wait .venv\Scripts\python.exe transcribe_folder.py "<your-folder>"
```

The empty `""` is the window title. `start` treats the first quoted item as a title, so without it, `start` would take your first quoted path as the title instead. `/wait` waits for the job to finish. `/belownormal` is a middle ground.

10.2 PowerShell

```
cd C:\Tools\faster-whisper
$py = '.\.venv\Scripts\python.exe'
$jobArgs = 'transcribe_folder.py', '"<your-folder>"'
$p = Start-Process $py -ArgumentList $jobArgs -NoNewWindow -PassThru
$p.PriorityClass = 'BelowNormal'
$p.WaitForExit()
```

Use `'Idle'` instead of `'BelowNormal'` to run only when the PC is otherwise idle. Keep the inner double quotes around the folder: `Start-Process` needs them for paths with spaces.

You can also change a running job's priority in **Task Manager**: on the **Details** tab, right-click `python.exe` and choose **Set priority**.

On macOS and Linux, start it with `nice -n 19 python transcribe_folder.py "<your-folder>"`.

---

11. Privacy and Copyright

This is general information, not legal advice. Copyright law differs between countries.

Downloading

- **Only download videos you're allowed to.** That means your own uploads, videos the creator or rights holder has given you permission to download, or videos the site itself lets you download.
- **YouTube's terms** forbid downloading except where YouTube provides a way to, or "with prior written permission from YouTube and, if applicable, the respective rights holders". They also forbid accessing the service by automated means. yt-dlp being legal to install doesn't change what a site's terms allow.
- **For your own YouTube videos,** use YouTube Studio's download or Google Takeout (Section 4.6).

Transcripts

- **A transcript of someone else's video is a copy of their words.** It's fine for your own private notes and research. Publishing it, posting it or sharing it widely is copying their work. The same rule applies to OCR text from books: see PDF and Document Tools.
- **Keep transcripts out of public repositories.** If your tools folder is a Git repository, add your output folder to `.gitignore` (for example, a line reading `Transcripts/`). Connecting a Custom Domain to Firebase App Hosting shows how `.gitignore` works.
- **When you quote,** keep it short, say who said it, link to the original, and check the quote against the audio. Whisper makes mistakes, and it can occasionally invent words in silence or music.
- **Recordings of private conversations** may need everyone's consent before you record or transcribe them, depending on where you live.

Privacy

- Everything in Sections 5 to 10 runs on your PC. The only internet use is the one-time model download (and yt-dlp's downloads).

- The audio files and transcripts stay wherever you saved them. Delete them when you no longer need them.

---

12. Troubleshooting

What you see	Why	What to do
`TypeError: open() got an unexpected keyword argument 'metadata_errors'`	PyAV 19 removed that option, and faster-whisper 1.2.1 still passes it	Run `python -m pip install "av<19"` in the faster-whisper venv, or use the Section 9 script, which reads audio itself. A faster-whisper release after 1.2.1 should fix it
Another error while opening or decoding a file	The file has no audio track, is damaged, or uses an unusual format	Check it plays. Decode it with PyAV as in Section 9, or convert it first with ffmpeg: `ffmpeg -i input.webm -vn -ac 1 -ar 16000 output.wav`
`yt-dlp` or `python` **is not recognized**	The venv isn't turned on in this window	Run `.\.venv\Scripts\Activate.ps1` in the tool's folder, or type the full path to the `.exe`
Requested format is not available	The video has no `m4a` audio	Add a fallback: `-f "bestaudio[ext=m4a]/bestaudio"`. Use `-F` to see what's offered
ffmpeg not found. The downloaded format may not be the best available	ffmpeg isn't installed	Harmless for audio-only downloads. Install ffmpeg if you want merged video
writing DASH m4a. Only some players support this container	Fixing YouTube's DASH audio needs ffmpeg	Harmless for faster-whisper. Install ffmpeg if a media player won't open the file
You have requested merging of multiple formats but ffmpeg is not installed	The format you chose needs a video and an audio stream merged	Choose an audio-only format (Section 4.2), or install ffmpeg
YouTube warns about a **JavaScript runtime**, or lists very few formats	yt-dlp-ejs or a JavaScript runtime is missing	Install with `"yt-dlp[default]"` and install Deno, or add `--js-runtimes node` (Section 3.1)
A download that used to work **now fails**	The site changed	Update yt-dlp (Section 3.2)
It downloads a **whole playlist**	The link includes `&list=`	Add `--no-playlist`
`%(title)s` comes out **mangled in a `.bat` file**	Command Prompt treats `%` specially in batch files	Write `%%(title)s` in batch files
The first run **sits there with no output**	It's downloading the model	Wait a few minutes. Check your internet connection and free disk space
A warning about **symlinks** in the Hugging Face cache	Windows doesn't allow symlinks by default	Harmless. Turn on Developer Mode, or set `HF_HUB_DISABLE_SYMLINKS_WARNING=1`
An error about a missing `cublas64_12.dll` or **cuDNN** file	The NVIDIA libraries for GPU use aren't installed or aren't on `PATH`	Install cuBLAS and cuDNN for CUDA 12 (Section 8.2), or use `device="cpu"`

What you see	Why	What to do
The same sentence **repeats** over and over, or text appears during silence or music	Whisper can loop or invent text	Use `vad_filter=True` and `condition_on_previous_text=False` (Section 9)
The transcript is **gibberish or in the wrong language**	An English-only (`.en` or distil) model on other languages, or a wrong guess	Use a multilingual model and set `language` (Section 7)
It's **very slow**	A large model on a CPU	Use `distil-small.en` or another smaller model, keep `int8`, and try `beam_size=1` (Section 8.3)
The PC is **sluggish** while it runs	The job is using every core at normal priority	Run it at low priority (Section 10)

---

13. Sources

yt-dlp

- yt-dlp README (options, format selection, output templates, dependencies)
- Installation wiki (pip and winget)
- EJS wiki (JavaScript runtimes for YouTube)
- Deno installation

faster-whisper and models

- faster-whisper README (install, usage, GPU, benchmarks)
- faster-whisper on PyPI (release history)
- faster-whisper model names (`utils.py`)
- `WhisperModel` and `transcribe` options (`transcribe.py`)
- Pull request #1495: PyAV 19 fix
- PyAV changelog (19.0.0 removes `metadata_errors`)
- OpenAI Whisper README (model sizes, `.en` models)
- Distil-Whisper
- Hugging Face Hub cache

Windows

- `start` command
- `ProcessPriorityClass` values

Terms

- YouTube Terms of Service
- Download YouTube videos that you've uploaded

Latest version: github.com/ImmuneMoon/Transcribing-Videos-with-yt-dlp-and-faster-whisper

If this helped, you can support Fulllion Creative Works through PayPal or Ko-fi.

© 2026 Fulllion Creative Works. All rights reserved.